

PROJET : Gestionnaire de Budget Personnel

Les membres dans ce projet dans GRP N°1 INFO :

- *FATIMA EZZAHRA ABOUNACHIT ->Membre 1*
- SALMA BELAICHA ->Membre 2
- Laila Borrou ->Membre 3

1. Introduction

Ce projet consiste à développer une application web permettant aux utilisateurs de gérer leurs finances personnelles en suivant leurs revenus et dépenses. L'application offre des fonctionnalités de base comme l'ajout, la modification et la suppression de transactions, ainsi que des rapports financiers sous forme de graphiques et de résumés mensuels.

2. Objectifs Réalisés

2.1 Interface Utilisateur (UI)

- **Ajout/Modification/Suppression de transactions** : L'utilisateur peut :
 - Ajouter une nouvelle transaction (revenu ou dépense) avec un montant, une catégorie et une date
 - Modifier une transaction existante.
 - Supprimer une transaction.
- **Affichage des transactions** : Une liste triée par date permet de visualiser l'historique.

- **Graphiques et Résumés** : Un graphique (camembert ou barres) montre la répartition des dépenses par catégorie.

2.2 Fonctionnalités Implémentées

- **Calcul du solde restant** : Le solde est mis à jour dynamiquement en fonction des entrées et sorties d'argent.
- **Historique mensuel** : Filtrage des transactions par mois avec un récapitulatif (total des revenus, dépenses et solde).
- **Validation des entrées** : Vérification que le montant est un nombre valide et que les champs obligatoires sont remplis.
- **Stockage des données** : Utilisation de `localStorage` pour sauvegarder les transactions entre les sessions.

2.3 Technologies Utilisées

- **Frontend** : HTML, CSS, JavaScript.
- **Stockage** : `localStorage` pour persister les données localement.
- **Visualisation** : Bibliothèque Chart.js (ou équivalent) pour les graphiques

3. Structure du Projet

3.1 organisation entre les membres :

1. Authentification = Membre 1 :

Rôle : Gestion des comptes utilisateurs.

Fichiers :

- **Frontend** :
 - `auth.js` (formulaires login/register)
 - `index.html` (page d'accueil)
- **Backend** :
 - `clsAuth.js` (logique métier)
 - `clsUserDataAccess.js` (accès DB)

- **Base de données :**

- Table `users` dans `schema.sql`

2. Budgets -> Membre 2 :

Rôle : Gestion des budgets mensuels.

Fichiers :

- **Frontend :**

- Sections "Budget" dans `dashboard.html/dashboard.js`

- **Backend :**

- `clsBudget.js` (*logique métier*)
- `clsBudgetDataAccess.js` (*accès DB*)

- **Base de données :**

- Table `budgets` dans `schema.sql`

3. Transactions -> Membre 3 :

Rôle : Gestion des revenus/dépenses.

Fichiers :

- **Frontend :**

- Sections "Transactions" dans `dashboard.html/dashboard.js`

- **Backend :**

- `clsTransaction.js` (*logique métier*)
- `clsTransactionDataAccess.js` (*accès DB*)

- **Base de données :**

- Tables `transactions`, `expenses`, `incomes` dans `schema.sql`

3.2 Fonctions Clés en JavaScript

1. Ajout d'une transaction :

```
function addTransaction(amount, category, type, date) {  
    // Validation + ajout au tableau et localStorage  
}
```

2. Calcul du solde :

```
function updateBalance() {  
    // Calcule le total des revenus - dépenses  
}
```

3. Affichage du graphique :

```
function renderChart() {  
    // Utilise Chart.js pour afficher les dépenses par catégorie  
}
```

4. Difficultés Rencontrées et Solutions

4.1 Problème : Gestion des Dates

- **Problème** : Filtrage des transactions par mois.
- **Solution** : Utilisation de `Date` en JavaScript et comparaison des mois/années.

4.2 Problème : Mise à Jour Dynamique du Graphique

- **Problème** : Le graphique ne se rafraîchissait pas après une nouvelle transaction.
- **Solution** : Détruire l'ancien graphique avant d'en créer un nouveau.

4.3 Problème : Validation des Entrées

- **Problème** : L'utilisateur pouvait entrer un montant négatif pour une dépense.
- **Solution** : Conversion en valeur absolue (`Math.abs()`).

5. Améliorations Possibles

- **Export des données** : Permettre d'exporter l'historique en CSV/PDF.
- **Synchronisation cloud** : Utiliser Firebase ou une API pour sauvegarder en ligne.
- **Catégories personnalisées** : Permettre à l'utilisateur d'ajouter ses propres catégories.
- **Notifications** : Alertes en cas de budget dépassé.

6. Conclusion

Ce projet a permis de mettre en pratique les concepts fondamentaux du développement web (HTML/CSS/JS) tout en intégrant une logique de gestion de données via `localStorage`. L'application est fonctionnelle, intuitive et prête à être étendue avec des fonctionnalités avancées.

Points Forts :

- Interface simple et efficace.
- Visualisation claire des finances.
- Persistance des données.

Axes d'Amélioration :

- Expérience mobile (responsive design).
- Gestion des erreurs plus robuste.